

⏳

Examen 2021

Ejercicio 5. Una cuenta de ahorros es compartida por varias personas. Cada persona puede depositar o retirar fondos de la cuenta. El saldo actual de la cuenta es la suma de todos los depósitos menos la suma de todos los reintegros. El saldo nunca puede ser negativo. Queremos usar un monitor para resolver el problema. El monitor debe tener 2 procedimientos: `depositar(c)` y `retirar(c)`. Suponer que los argumentos de las 2 operaciones son siempre positivos, e indican las cantidades a depositar o retirar. El monitor usará la semántica señalar y espera urgente (SU).

El reintegro de fondos a los clientes se hace únicamente según el orden de llegada, es decir, si hay más de un cliente esperando, sólo el primero que llegó puede optar a retirar la cantidad que desea, mientras esto no sea posible, esperarán todos los demás clientes, independientemente de cuanto quiera retirar cada uno. Por ejemplo, suponer que el saldo es 200 unidades y un cliente está ya esperando un reintegro de 300 unidades; si llega después otro cliente que quiere retirar las 200 unidades, debe esperarse. Para resolverlo se pueden utilizar variables condición prioritarias.

```
monitor Cuenta:
    private:
        int saldo,posicion;
        condition cola;
```

```
    public:
        Cuenta();
        depositar(int c);
        returar (int c);

    Cuenta::Cuenta(){
        saldo = 0;
        posicion = 0;
     }

     Cuenta::depositar(int c){
         saldo+=c;
         cola.signal();
     }

     Cuenta::retirar(int c){
         int pos = posicion;
         posicion++;
         if (cola.queue()){
             cola.wait(pos);
         }

         while (saldo < c){
                 cola.queue(pos);
             }
             saldo-=c;

             cola.signal();
         }
     }
```

Ejercicio 3. Considerando el siguiente programa concurrente:

```
1  S :: cobegin <x = x+2> || <x = x+3> || <x = x+4> coend
```

Aplicando las reglas de demostración concurrentes estudiadas, demostrar que el siguiente triple de Hoare es un aserto demostrablemente cierto de la Lógica de Programas:

$$\{x = 0\}\ S\ \{x = 9\}$$

Aplicando el axioma de asignación llegamos a las siguientes tuplas:

```
{x=0 v x=3 v x=4 v x=7} <x=x+2> {x=2 v x=5 v x=6 v x=9}
{x=0 v x=2 v x=4 v x=6} <x=x+3> {x=3 v x=5 v x=7 v x=9}
{x=0 v x=2 v x=3 v x=5} <x=x+4> {x=4 v x=6 v x=7 v x=9}
```

Así, debemos demostrar la no interferencia de todas las precondiciones y postcondiciones entre ellas, quedando las siguientes combinaciones que demostrar:

```
NI(P2,S1)  NI(P3,S1)  NI(Q2,S1)  NI(Q3,S1)
NI(P1,S2)  NI(P3,S2)  NI(Q1,S2)  NI(Q3,S2)
NI(P1,S3)  NI(P2,S3)  NI(Q1,S3)  NI(Q2,S3)
```

Vamos a desarrollar NI(P2,S1)

```
NI(P2,S1) = {P2 intersection P1} S1 {P2} = {x=0 v x=4} x=x+
2 {x=0 v x=2 v x=4 v x=6}
```

Como podemos esto es cierto por el axioma de asignación, NI(P2,S1) es cierto por la primera regla de la consecuencia.
Voy a obviar la demostración del resto de NI.

Así, aplicando la regla de la composición concurrente llegamos al siguiente triple:

```
{P1 intersección P2 interseccion P3} S {Q1 interseccion Q2
interseccion Q3}

{x=0} S {x=9}
```